

CAMINO DE SANTIAGO · PRODUCTO

Chat multiidioma *en tiempo real*

Cómo TuCamino traduce conversaciones entre peregrinos, albergues y negocios sin que nadie note que hay una traducción de por medio — resumen técnico de la arquitectura detrás de la funcionalidad.

TuCamino — tucamino.app · Fernando Fernández González · Full Stack Developer

01

La idea central

Cada conversación —ya sea un canal público de etapa, un canal de albergue o un chat 1:1— sabe qué idiomas hablan realmente sus participantes. No se traduce "por traducir": se traduce solo lo necesario, para quien lo necesita.

Eso significa que un canal abierto a cientos de peregrinos se comporta de forma distinta a una conversación privada entre dos personas. El sistema distingue ambos casos y ajusta el alcance de la traducción según el contexto, sin que el usuario tenga que configurar nada.

02

Arquitectura de datos

El modelo gira en torno a un puñado de tablas en Postgres, vía Supabase: una tabla de **conversaciones** que puede representar distintos tipos de hilo (canal de etapa, canal de albergue o negocio, DM), una tabla de **participantes** que resuelve la relación N:M de quién puede leer y escribir en cada una, y una tabla de **mensajes** donde las traducciones se guardan como un campo estructurado que empieza vacío y se rellena de forma asíncrona una vez procesado el mensaje.

Esa separación —participantes como entidad propia, en lugar de idiomas hardcoded por conversación— es lo que permite que el sistema sepa "a quién traducir qué" sin necesidad de configuración manual: basta con consultar quién forma parte de esa conversación en concreto.

03

Arquitectura de archivos y capas

El proyecto combina dos formas de acceso a los datos, cada una con su función: una API construida con **Hono** se encarga de la lógica que requiere reglas de negocio (resolver o crear canales, controlar límites de uso, disparar traducciones), mientras que la lectura y el envío de mensajes en sí van directos del cliente a Supabase, protegidos por políticas de seguridad a nivel de fila (**RLS**).

No es una arquitectura 100% API-first ni 100% BaaS puro, sino una mezcla deliberada: la API entra en juego donde hace falta orquestar o validar, y se sale de en medio donde Supabase ya resuelve el problema mejor — tiempo real, seguridad de acceso.

04

Supabase como columna vertebral del pipeline

Cuando se guarda un mensaje nuevo, un **webhook de base de datos** se dispara automáticamente sobre ese evento y arranca el proceso de traducción, sin que el backend tenga que estar "escuchando" activamente. Una vez traducido, se actualiza ese mismo registro, y **Supabase Realtime** propaga el cambio a cada participante conectado al instante — no hay refrescos, no hay polling, el mensaje simplemente aparece traducido.

Esta combinación de webhooks + tiempo real es lo que permite que todo el sistema se sienta instantáneo sin tener que montar infraestructura propia de mensajería desde cero.

05

La IA que traduce

La traducción no la hace un motor de traducción literal tipo diccionario. Se apoya en la **API de Claude**, de Anthropic, con un modelo pensado para tareas rápidas y ligeras (**Haiku**), orientado a mantener un tono conversacional natural y a respetar nombres propios de lugares y albergues en lugar de traducirlos — algo que un traductor automático clásico suele estropear.

06

Lo que hace que esto no sea "solo traducir texto"

- Un canal público y un chat privado no funcionan igual por debajo, aunque para el usuario se vean idénticos.
- El sistema tiene que decidir, en cada conversación, a quién le corresponde qué traducción — y eso no es tan trivial como parece cuando una misma persona participa en varios tipos de canal a la vez.
- Hay decisiones de arquitectura pensadas específicamente para que esto escale sin disparar costes de IA según crece la comunidad.

Si os interesa el detalle de cómo está montado el webhook, por qué Claude y no otro modelo, o cómo se reparte la lógica entre la API y Supabase, preguntadme sin problema.

TuCamino — Vive el Camino, en tu idioma.